Search

Xuanji Yu

Courtesy of Professor Marie Roch

Chapter 3, Russell & Norvig

Border Tuner by Rafael Lozano-Hemmer
international searchlight art installation,
El Paso, TX y ciudad Juárez, Chihuahua
Photo credit: Mariana Yañez

Solving problems through search

Systematically exploring possible **states** and actions to find an optimal or satisfactory path from an initial state to a goal state.

- State – atomic representation of world
  - states of the world are considered as wholes, with no internal structure visible to the problemsolving algorithms
- Goal formulation
  - What objective(s) are we trying to meet?
  - Can be represented as a set of states that meet objectives: goal states
- Problem formulation
  - Decide actions and states (plan) to reach a goal

Search

- Assume task environment is
  - fully observable
  - discrete (finite # of actions)
  - rules known
  - deterministic actions
    - so each action has exactly one outcome
  - episodic, single agent, static

author unknown

- Search process returns a plan:
  set of states & actions to reach a goal state
  - Plan can be executed

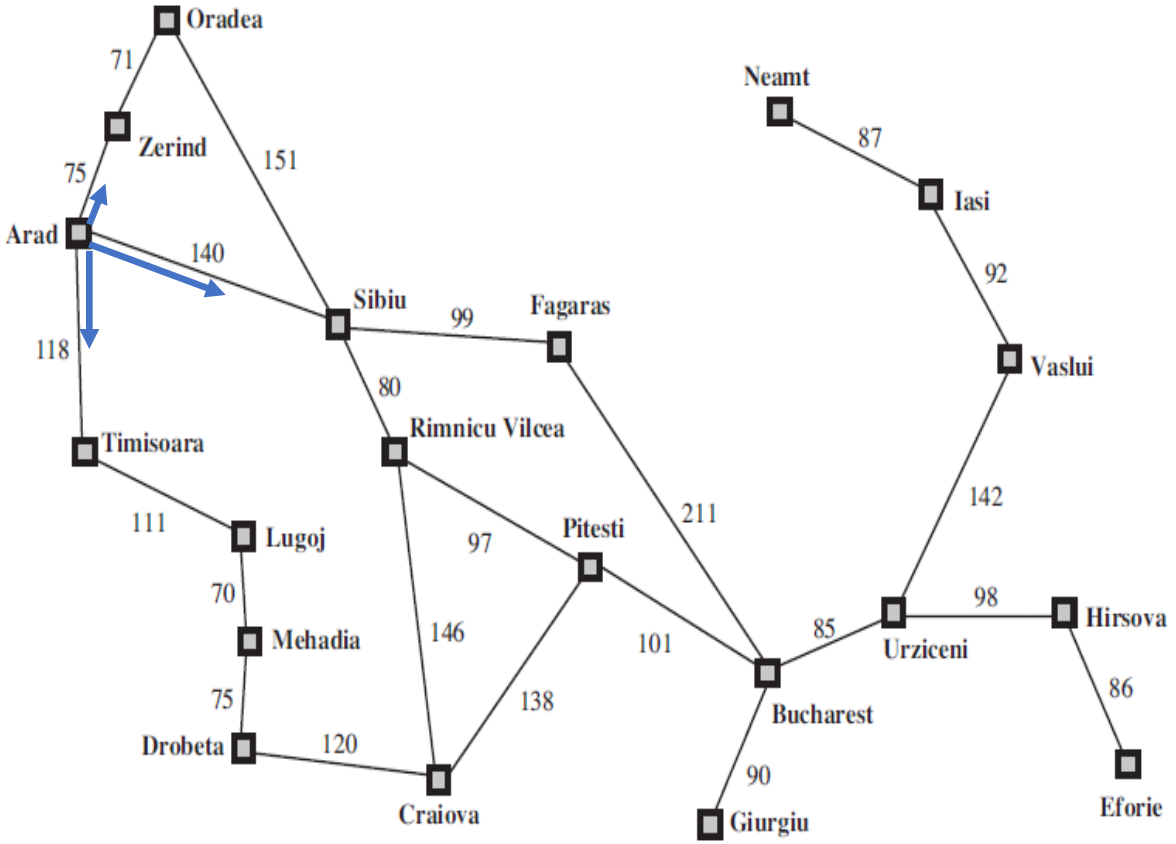
- Goal based, model based agent

Search problem components

- Initial state

Search problem components

- Initial state
- Actions
  - function that returns set of possible decisions from a given state
  - actions(in(arad)) →{go(sibiu), go(Timisoara), go(zerind)}



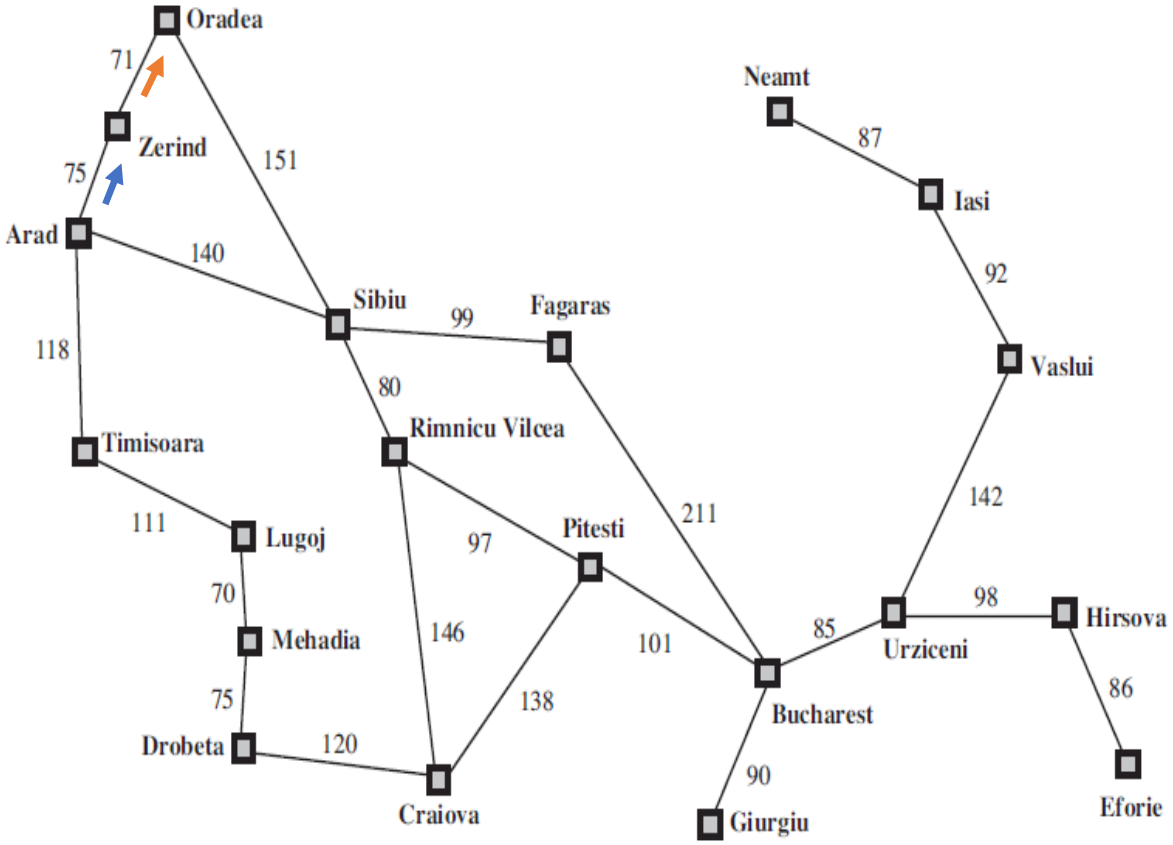
Abstract view of Romanian roads

(Russel and Norvig 2010, Fig 3.2)

Note: Abstractions are valid when we can map them onto a more detailed world

Search problem components

- Initial state
- Cost
  - Each action has a step cost:
    cost(in(arad), go(zerind), in(zerind)) = 75
  - A path has a cost which is the sum of its step costs:
    - path: in(arad), in(zerind), in(Oradea)
    - cost of path
      cost(in(arad), go(zerind), in(zerind)) + cost(in(zerind), go(oradea), in(oreadea)) = 75 + 71 = 146



Abstract view of Romanian roads

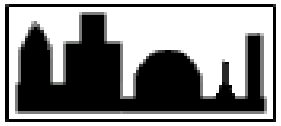
(Russel and Norvig 2010, Fig 3.2)

Search problem components

- Initial state
- Actions
- Cost
- Transition model is a function that reports the result of an action applied to a state:

result(in(arad),go(zerind)) →in(zerind)

Arad



Zerind

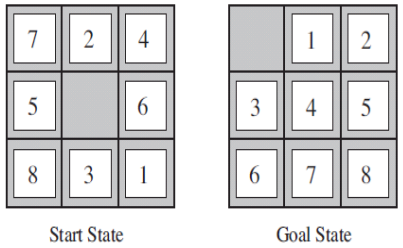
Search problem components

- Initial state
- Actions
- Cost
- Transition model
- Goal predicate
  - Predicates are expressions in logic that evaluate to true or false based on whether the current state of the system achieves the desired state.
    goal: {in(bucharest)}

Any path that reaches a goal is a *solution*, the lowest cost path is an *optimal solution*.

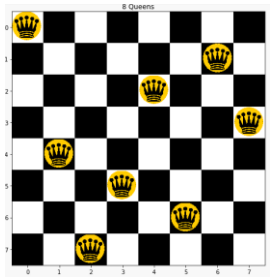
Sample toy problems

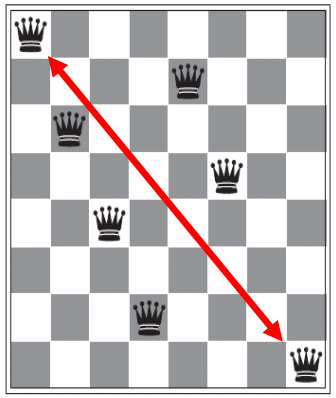
- n-puzzle
  - a sliding puzzle
  - goal: place the tiles in numerical order (from left to right, top to bottom)



8-puzzle and one possible goal state
[Figure 3.4 R&N 2010]

- n-queens
  - 8-queens: placing 8 chess queens on an 8×8 chessboard so that no two queens threaten each other
  - Goal: no two queens share the same row, column, or diagonal, e.g.,





8-queens state
[Figure 3.5 R&N 2010]

see text for other examples

Constructing a problem: n-queens

- States
  1. complete-state:
     - n-queens on board
     - move until no queen can capture another.
  2. Incrementally place queens
     - initial empty board
     - add one queen at a time

Incremental n-queens

- state: Any arrangement of [0,n] queens
- initial state: empty board
- actions: add queen to empty square
- transition model: new state with additional queen
- goal test: n queens on board, none can attack one another

Incremental n-queens

- A well-designed problem restricts the state space
  - Naïve 8 queens
    1st queen has 64 possibilities
    2nd queen has 63 possibilities…

$$64 \times 63 \times 62 \ldots \approx 1.8 \times 10^{14}$$

  - Smarter:
    - Actions only returns positions that would not result in capture
    - State space reduced to 2057 states.

Vacuum world

Search problem components

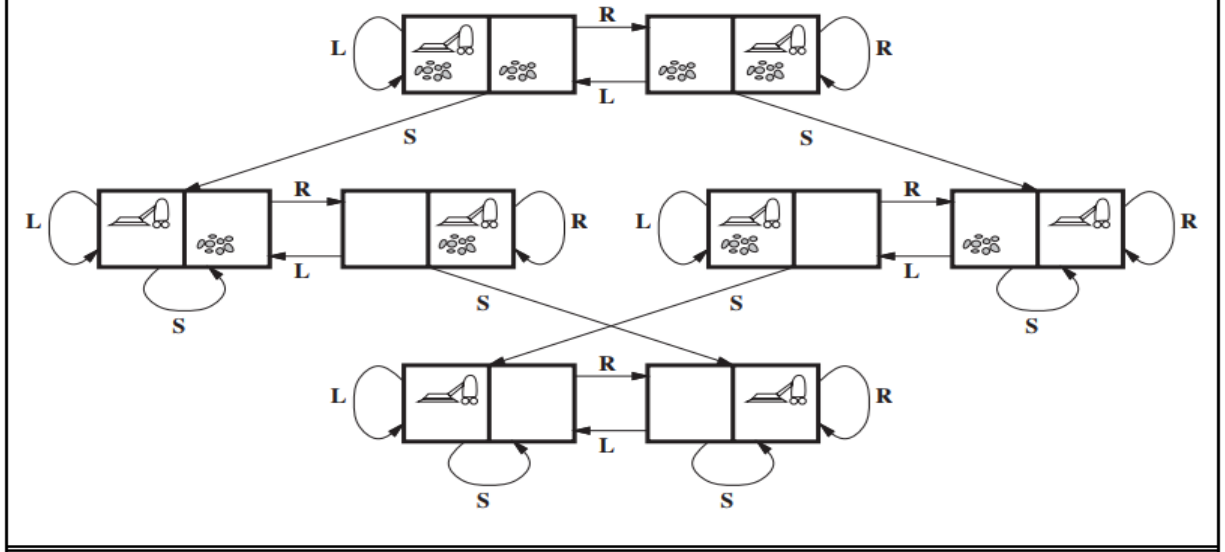
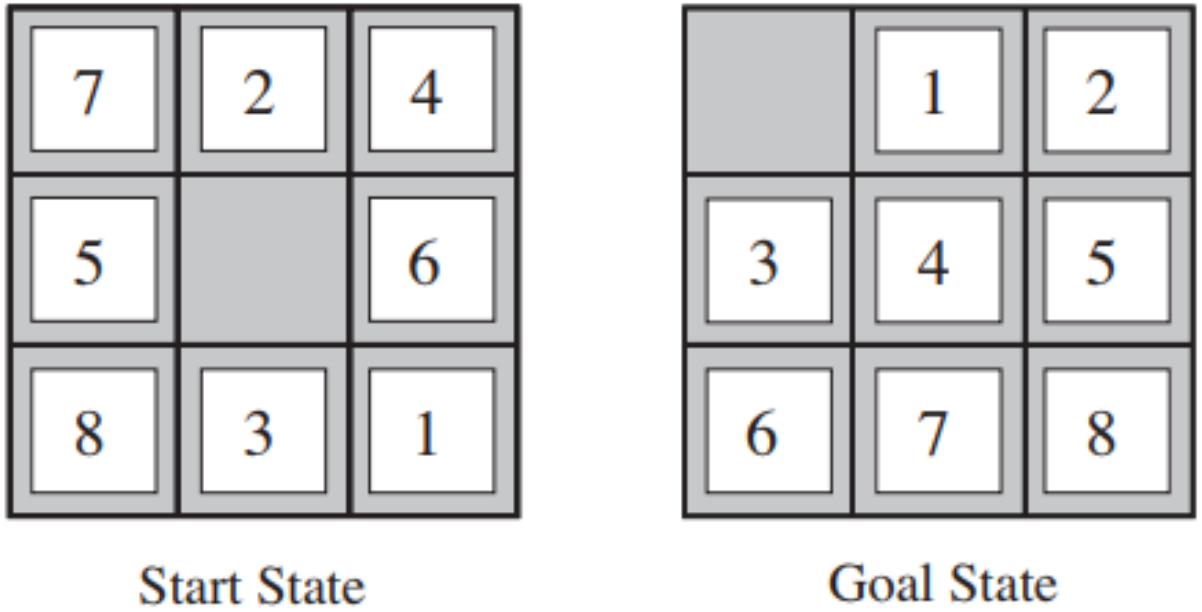


Figure 3.3 The state space for the vacuum world. Links denote actions: L = *Left*, R = *Right*, S = *Suck*.

- state: agent location and dirt locations
- initial state: any designated state
- actions: Left, Right, and Suck
- transition model: actions have their expected effects, see figure 3.3
- goal test: all the squares are clean

8-puzzle

Search problem components



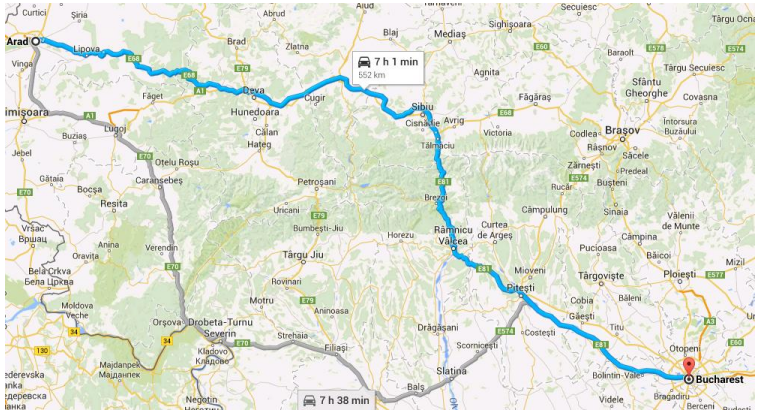
- state: location of each of the eight tiles and the blank
- initial state: any designated state
- actions: Movement of blank space Left, Right, Up, or Down
- transition model: Given a state and action, return the resulting state
- goal test: check whether state meet goal configuration

Classic real-world problems

- route-finding problem
  - transportation
    (car, air, train, boat, etc.)
  - networks
  - operations planning
- touring problem

  Visit a set of states ≥1 time
- traveling salesperson

  Visit a set of states exactly 1 time



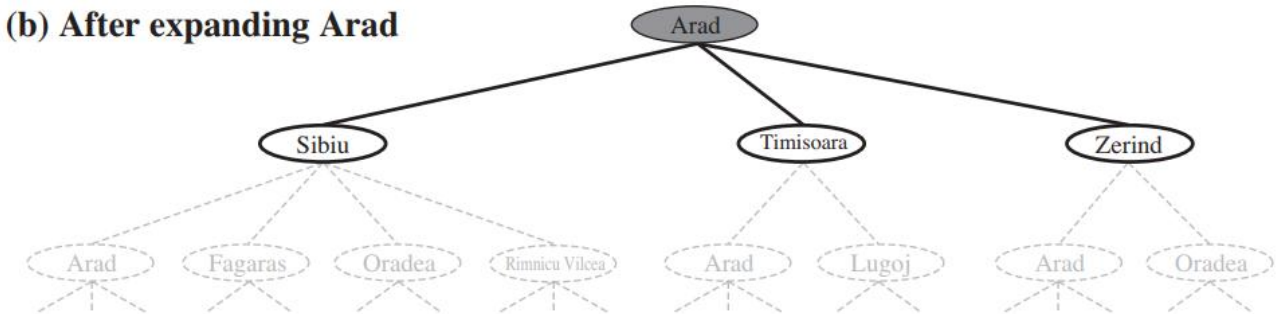
- Others: VLSI (Very Large-Scale Integration) layout, autonomous vehicle navigation & planning, (DNA) assembly sequencing, pharmaceutical discovery

Searching for solutions – Search tree

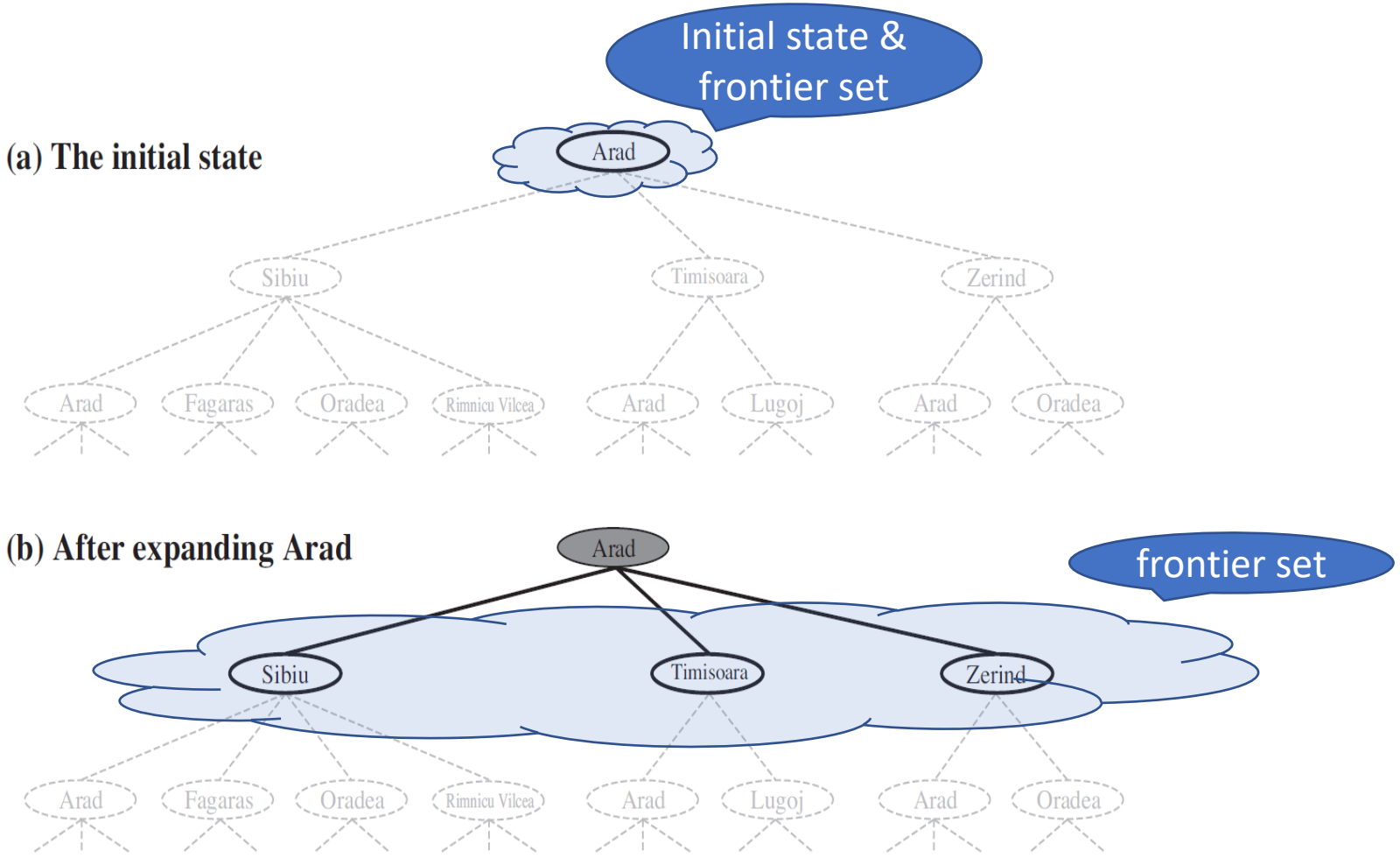
- Solution: an action sequence, starting at initial state

Search tree

- Node: state etc.
- Edge: transition to move from one state to another



Search trees

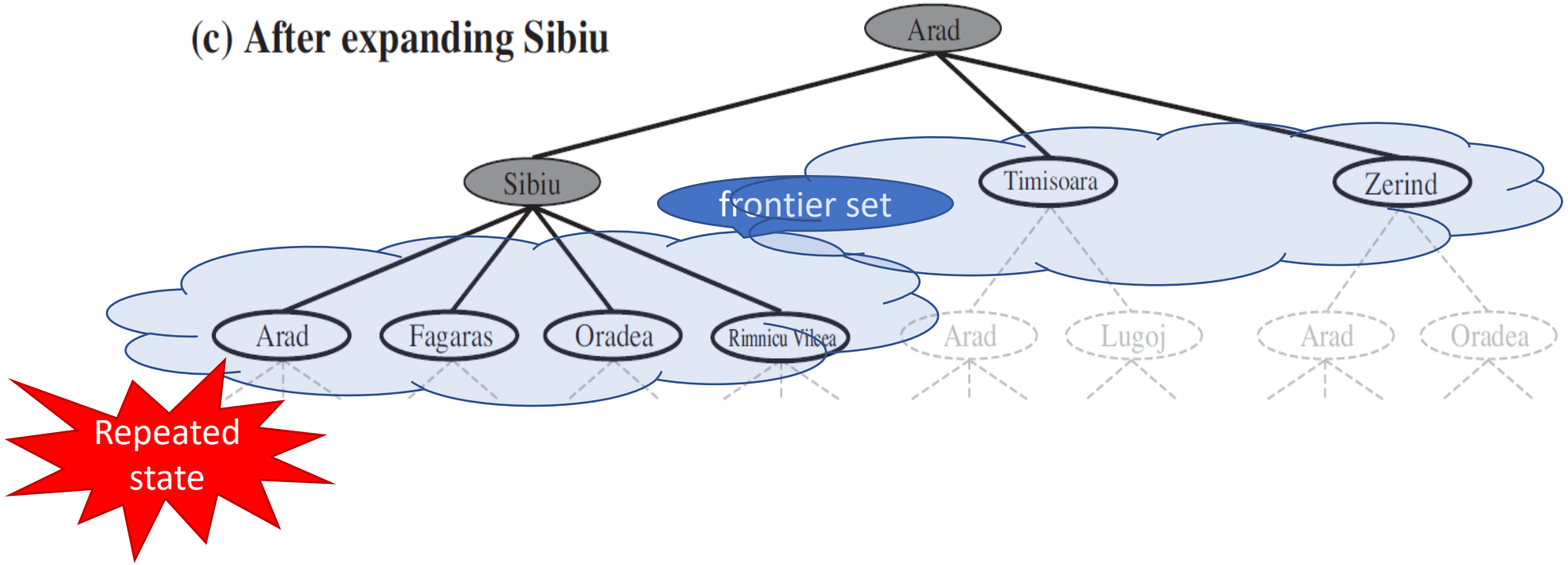


- Leaf node: node with no children in the tree
- Frontier Set: The set of all leaf nodes available for expansion at any given point

[Figure 3.6 R&N 2010]

frontier set also known as an open list or fringe set

Search trees



Search tree

- Frontier set* consists of leaf nodes
- Redundant paths occur when
  - ∃ (there is) more than 1 path between a pair of states
  - cycles in the search tree (loops) are a special case

* Frontier set is also known as the open list or fringe set.

Redundant paths

"Those who cannot remember the past are condemned to repeat it"

George Santayana,
Spanish-American philosopher 1863-1952

- Sometimes, we can define our problem to avoid cycles
  e.g. n-queens: queen must be placed in the leftmost empty column
- Otherwise: Explored set
  - Track states that have been investigated
  - Don't add any actions that have already occurred

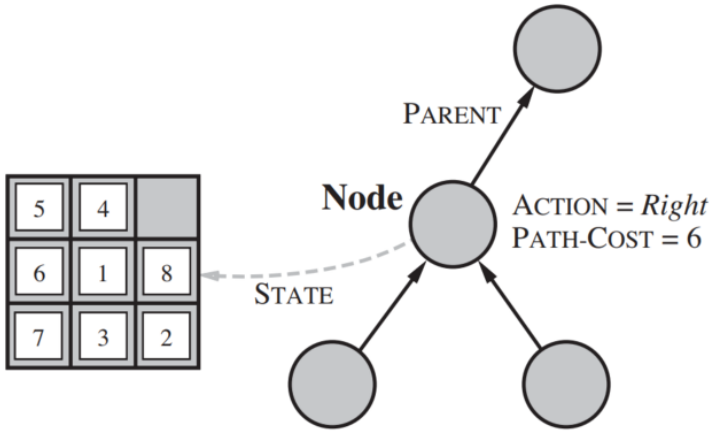
Tree Search algorithm (NO need to store explored set)

```
function tree-search(problem)
  frontier = problem.initial_state()
  done = found = False
  while not done
    node = frontier.get_node()  # remove state
    if node in problem.goals()
      found = done = True
    else
      frontier.add_nodes(results from actions(node))
      done = frontier.is_empty()
  return solution if found else return failure
```

Graph Search algorithm (explored set)

```
function graph-search(problem)
   frontier = problem.initial_state()
   done = found = False
   explored = {} # keep track of nodes we have checked
   while not done
      node = frontier.get_node()  # Remove a state from the frontier and process it
      explored = union(explored, node)
      if node in problem.goals()
         found = done = True
      else
         # only add novel results from the current node
         nodes = setdiff(results from actions(node), union(frontier,explored))
         frontier.add_nodes(nodes)
         done = frontier.is_empty()
   return solution if found else return failure
```

Search architecture – Node



- Node representation (Data structure)
  - state – current state of the problem to which node correspond
  - parent – ancestor in tree
    allows us to find the solution from a goal node by chasing pointers and reversing the path
  - action – Action on parent to generate this node
  - path-cost – What is the cost to reach this node from the tree's root. Usually denoted g(n).

Important: Nodes is a bookkeeping data structure used to represent the search tree. These are different from problem states, which is a configuration of the world. (Example: Tic-tac-toe)

Search architecture – Node

- Given components for a parent node
  - it is easy to compute the necessary components for a child node.

```
function child-node(problem, node, action)
  child.state = problem.result(node.state, action)  # result of an action applied to a state
  child.parent = node
  child.path_cost = node.path_cost +
    problem.cost(node.state, action, child.state)
  return child
```

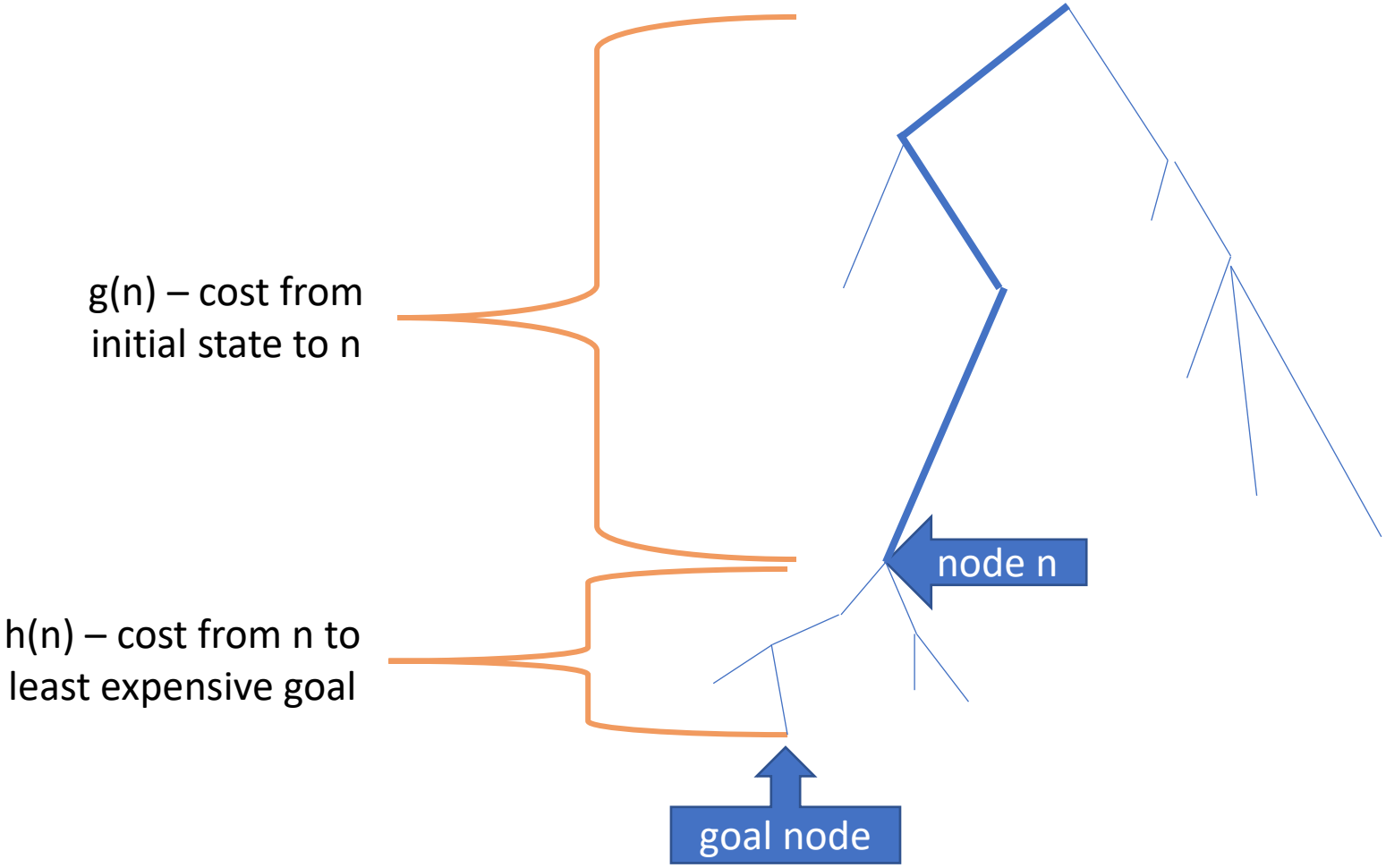
Search architecture – Data structure to store node

- Frontier set is usually implemented as a queue
  - First in, First out (FIFO) – traditional queue
  - Last in, First out (LIFO) – stack
  - priority queue

    Pop element of the queue with highest priority to some ordering function

- Explored set – Need to make states easily comparable
  - hash the state or
  - store in canonical form (e.g. sort visited cities for traveling salesperson problem)

Search architecture



g(n) *and h(n) are frequenty not known precisely.*
Estimates are denoted or $g'(n)$ & $h'(n)$ *or* $\hat{g}(n)$ & $\hat{h}(n)$

A generic graph search algorithm

```
function graph-search(problem)
   frontier = problem.initial_state()  # priority queue (lowest cost)
   done = found = False
   explored = {} # keep track of nodes we have checked
   while not done
      node = frontier.get_node()  # remove state
      explored = union(explored, node)
      if node in problem.goals()
         found = done = True
      else
         # only add novel results from the current node
         nodes = setdiff(results from actions(node), union(frontier,explored))
         for n in nodes
            n.cost = g'(n) + h'(n) # cost/estimate start→n + n→goal
         frontier.add_nodes(nodes)  # merge new nodes in by estimated cost
         done = frontier.is_empty()
   return solution if found else return failure
```

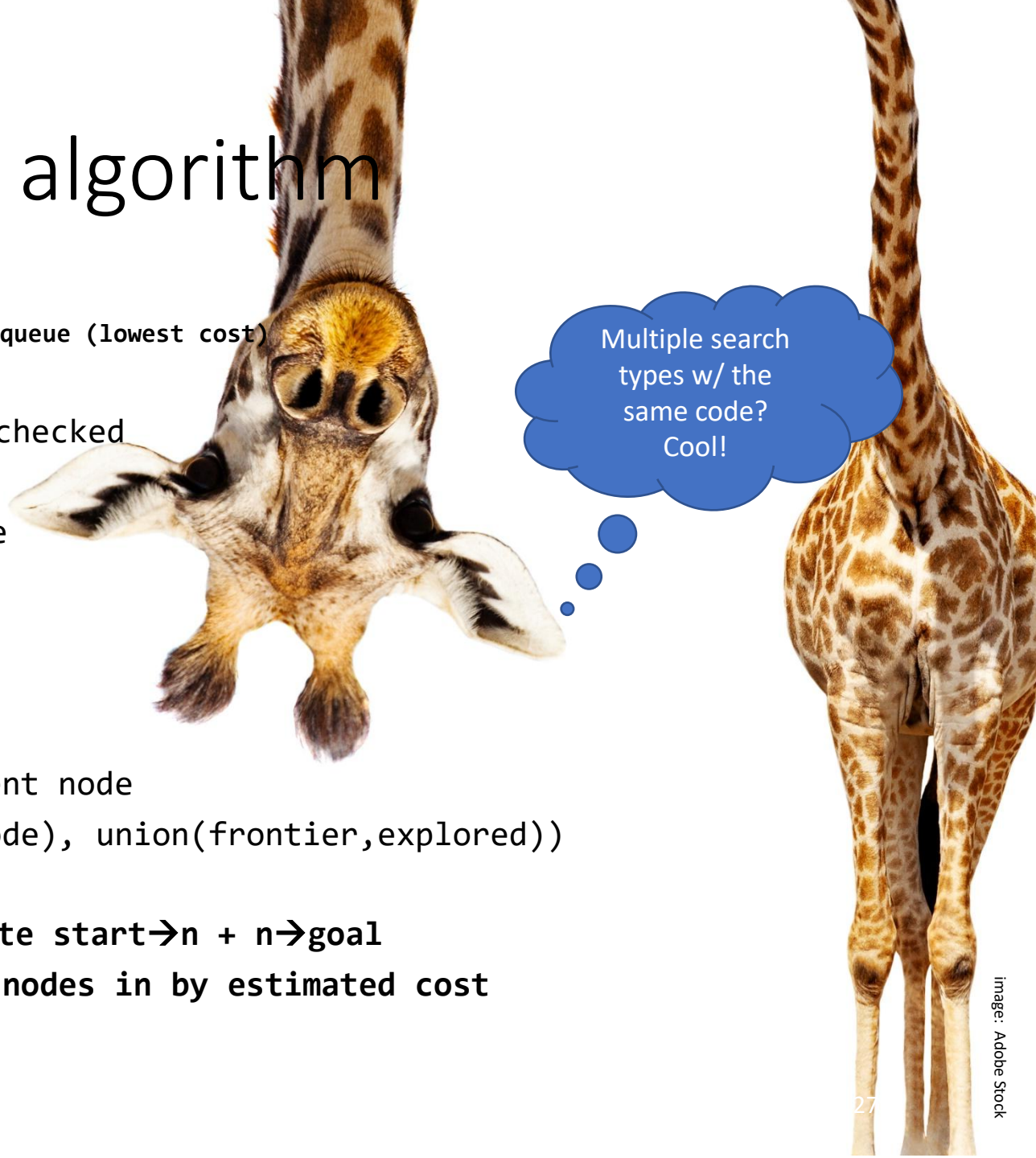


image: Adobe Stock

Questions to ask ourselves

Will a search be?

- complete – completeness guarantees to find a solution when one exists
- optimal – cheapest solution available as measured by the sum of costs of actions along the solution path

Uninformed (blind) search

- No awareness of whether or not a state is promising
- Strategies depend on order of node expansion
  - breadth-first
  - uniform-cost
  - depth-first
  - variants: depth-limited, iterative deepening, bidirectional

- Note: Text uses different queue types for frontier, with our generic search algorithm everything is a priority queue, smallest values first.

Breadth-first search

- Shallowest unexpanded node is chosen for expansion

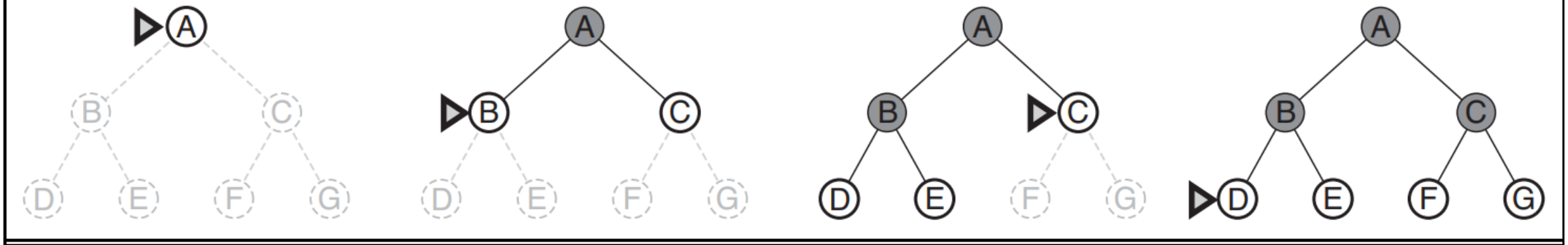


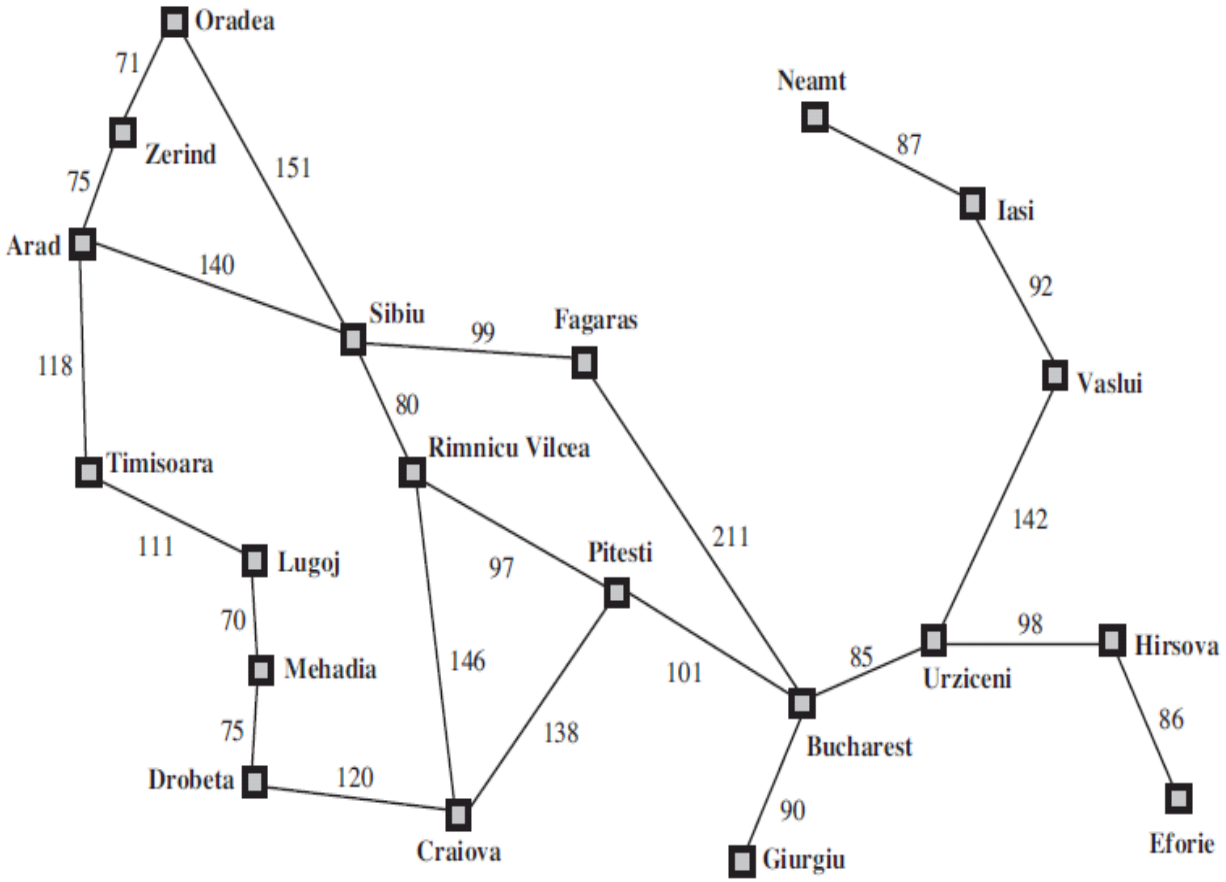
Figure 3.12 Breadth-first search on a simple binary tree. At each stage, the node to be expanded next is indicated by a marker.

Breadth-first search

For all n: $\forall n\ g'(n) = \text{depth}(n)$ and $h'(n)=0$ *(or any other constant k)*

Abstract view of Romanian roads

(Russel and Norvig 2010, Fig 3.2)



g(n) – cost from initial state to n

h(n) – cost from n to least expensive goal

Breadth-first search

- Guarantees
  - completeness – will find a solution if one exists
  - best (optimal) path *if cost is a nondecreasing f(depth)*

- How can we measure performance?
  - Time complexity
  - Space complexity

Complexity

- Measure of the number of operations (time) or memory (space) required
- Analysis of performance as the number of items n grows:
  - worst case
  - average case
- Example:

```
def foobar(n):
  x = 0
  for i in xrange(n):
    for j in xrange(n):
      x = x + i*i + j*j
  return x * x
```

There are $T(n)=4n^2+1$ arithmetic operations

Complexity

- We define "big oh" of n as follows:
  Arithmetic operations $T(n)$ is $O\big(f(n)\big)$ if $T(n) \leq kf(n)$
  for some k & $\forall n > n_0$
- Role of k and n_0
  Coefficients of highest order polynomial aren't relevant.
- Implications:
  - $T(n) = 4n^2+1 \rightarrow O(n^2)$
  - $T'(n) = 500n+8 \rightarrow O(n)$

  For some small values of n, T(n) is better, but as n increases T(n) will be worse. Using the big-oh notation abstracts this away and we know in general that the second algorithm is better.

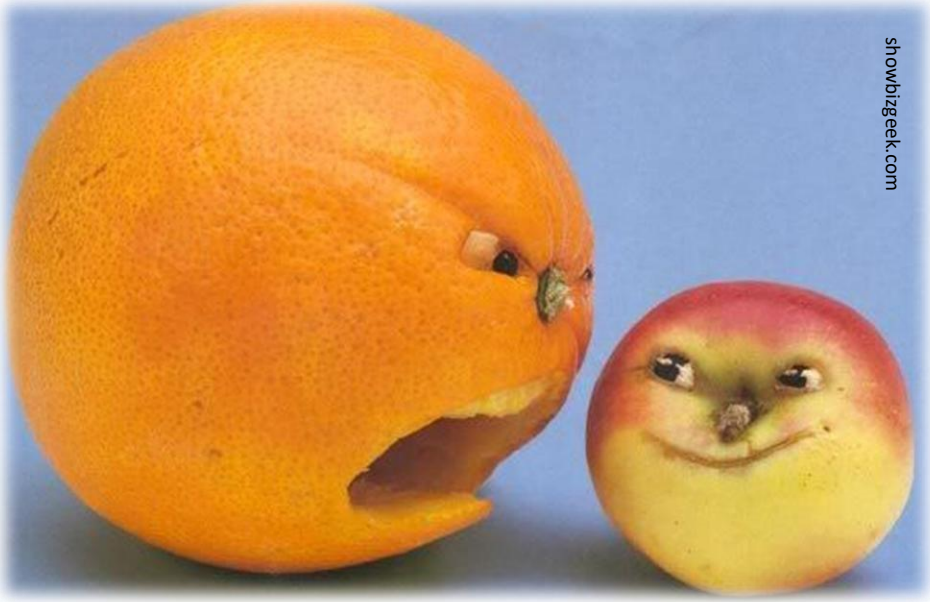
Search complexity

Measured with respect to search tree:

- Complexity is a function of
  - Branch factor – max # of successors
  - Depth of the shallowest goal node
  - Maximum length of a state-space path (number of edges or transitions from initial state to goal state)
- Time measurement: # nodes expanded
- Space measurement: maximum # nodes in memory

Search complexity

- “Search cost” – time complexity
- “Total cost” – time and space complexity
  Problematic to fuse the metrics…

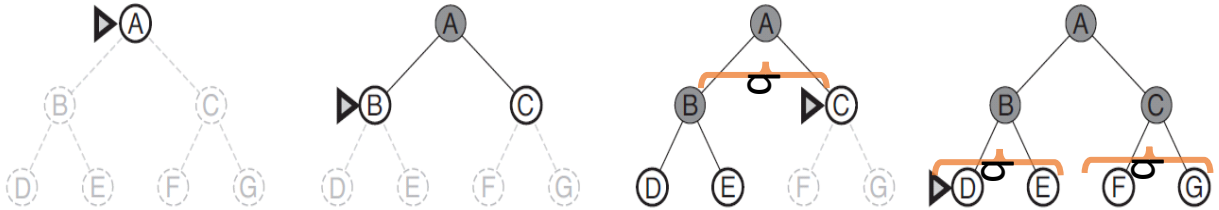


Breadth-first search performance

- Assume branch factor b (max # of successors)
- Time complexity:

  $b + b^2 + b^3 + \cdots + b^d = O(b^d)$ *

  Depth of the Shallowest Goal Node (d)
- Space complexity
  - Every generated node remains in memory, $O(b^{d-1})$ in explored and $O(b^d)$ in frontier.



Uniform-cost search

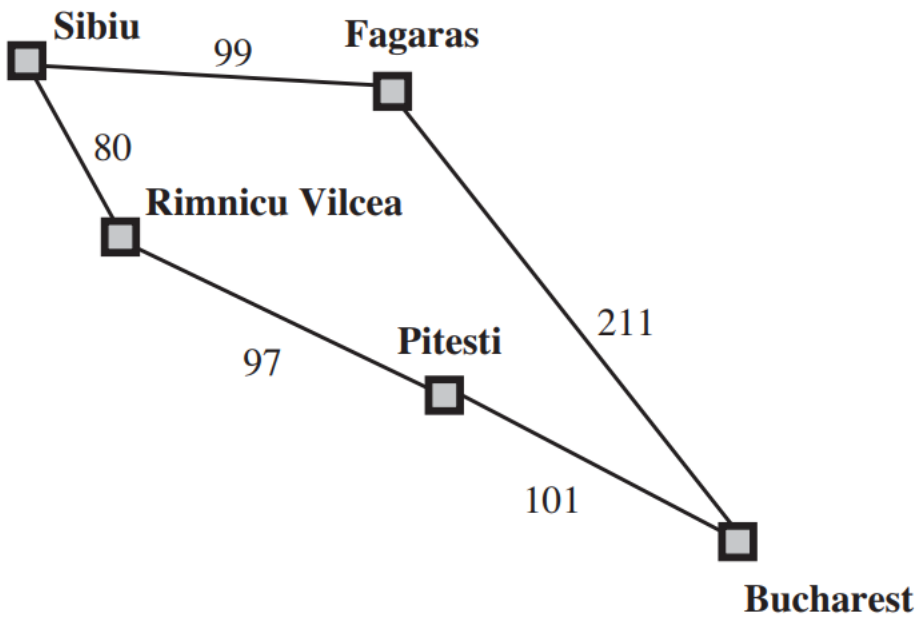
- Similar to breadth-first, g'(n) uses edge costs

  $\forall n\ g'(n) = cost(edge(parent \rightarrow n))$ and $h'(n) = k$

  - Spread out in waves of depth vs path-cost

Abstract view of Romanian roads

(Russel and Norvig 2010, Fig 3.2)



Sibiu to Bucharest

- 80 vs 99: 80
- 80+97 vs 99: 99
- 99+211 vs 80+97+101: 80+97+101

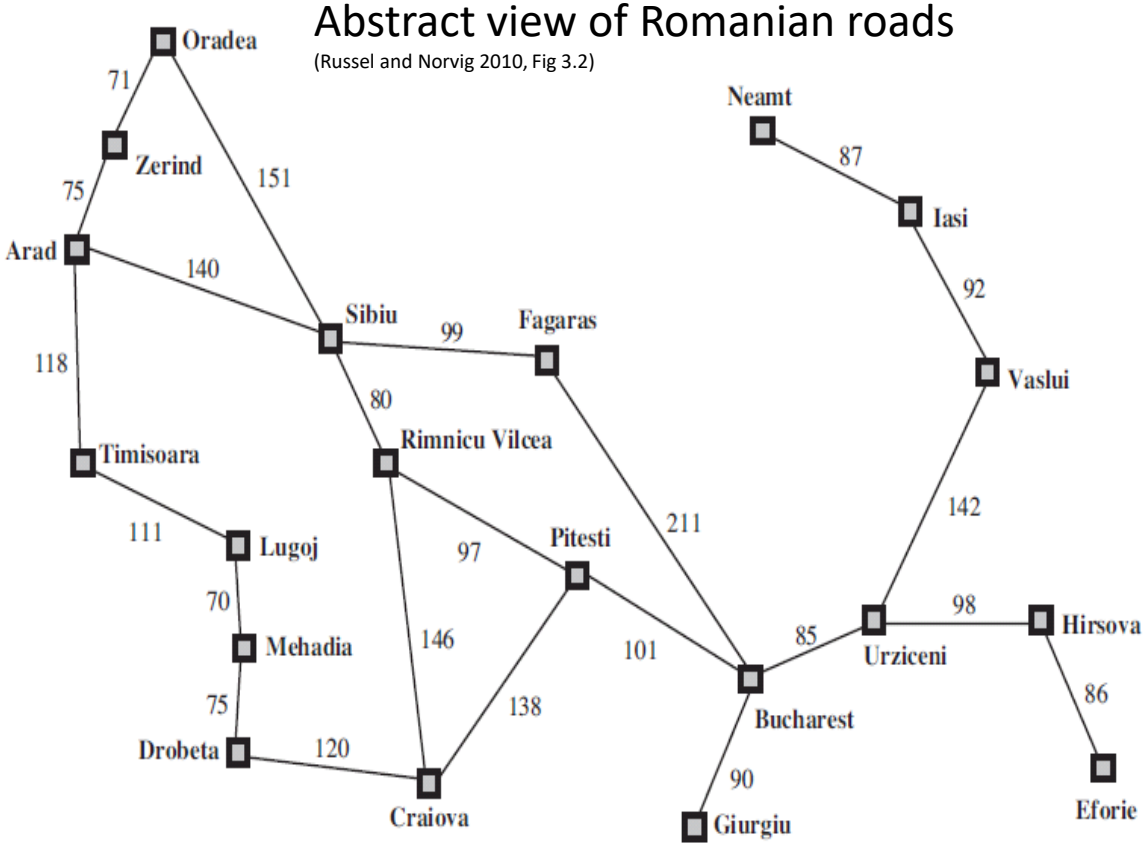
Uniform-cost search

- Similar to breadth-first, g'(n) uses edge costs

  $\forall n\ g'(n) = cost(edge(parent \rightarrow n))$ and $h'(n) = k$

- Nodes are expanded in order of optimal cost → optimal solution
- Complexity function of minimum cost for all actions

Abstract view of Romanian roads
(Russel and Norvig 2010, Fig 3.2)



Depth-first search

- Deepest node is expanded first

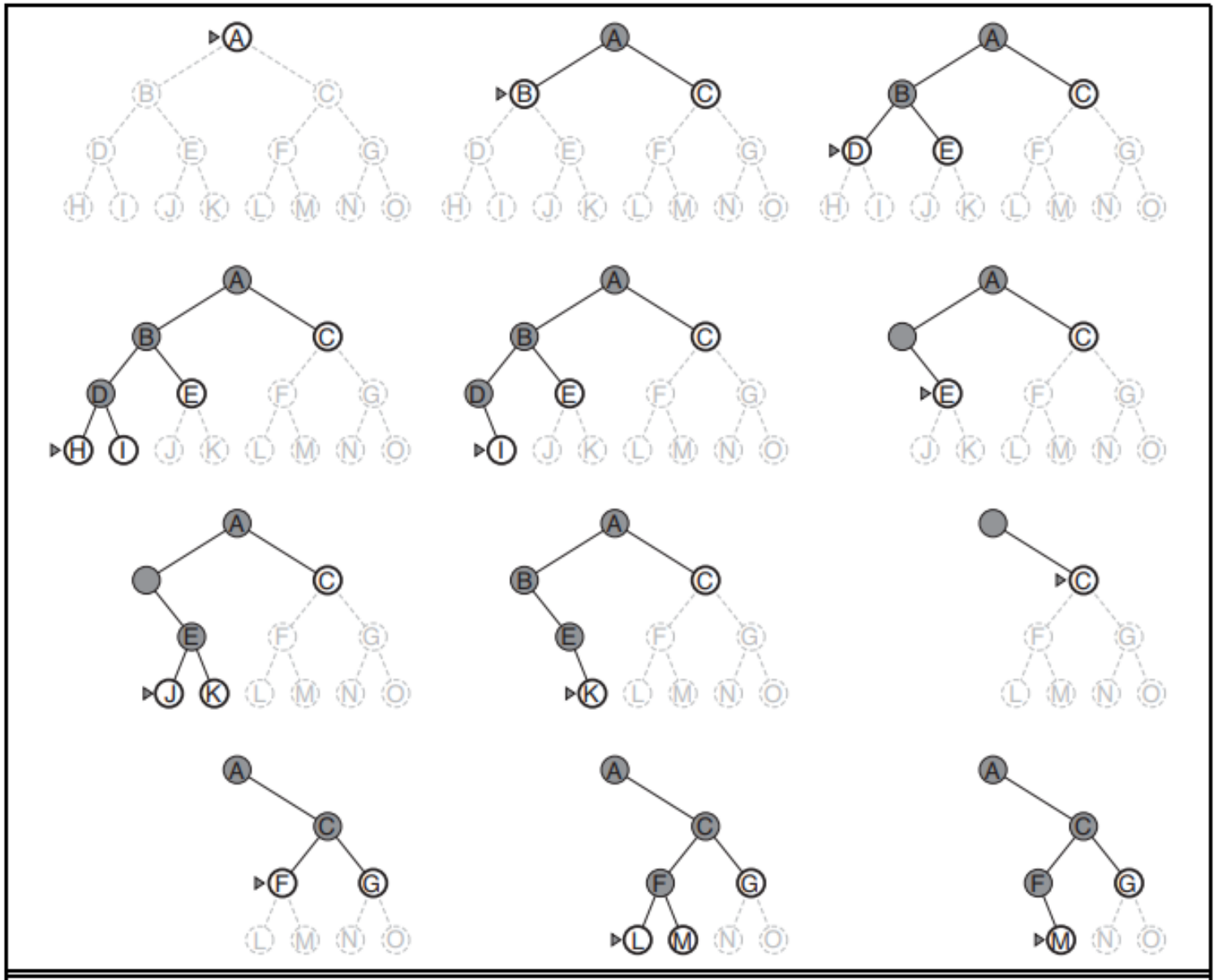


Figure 3.16 Depth-first search on a binary tree. The unexplored region is shown in light gray. Explored nodes with no descendants in the frontier are removed from memory. Nodes at depth 3 have no successors and M is the only goal node.

Depth-first search

- Deepest node is expanded first

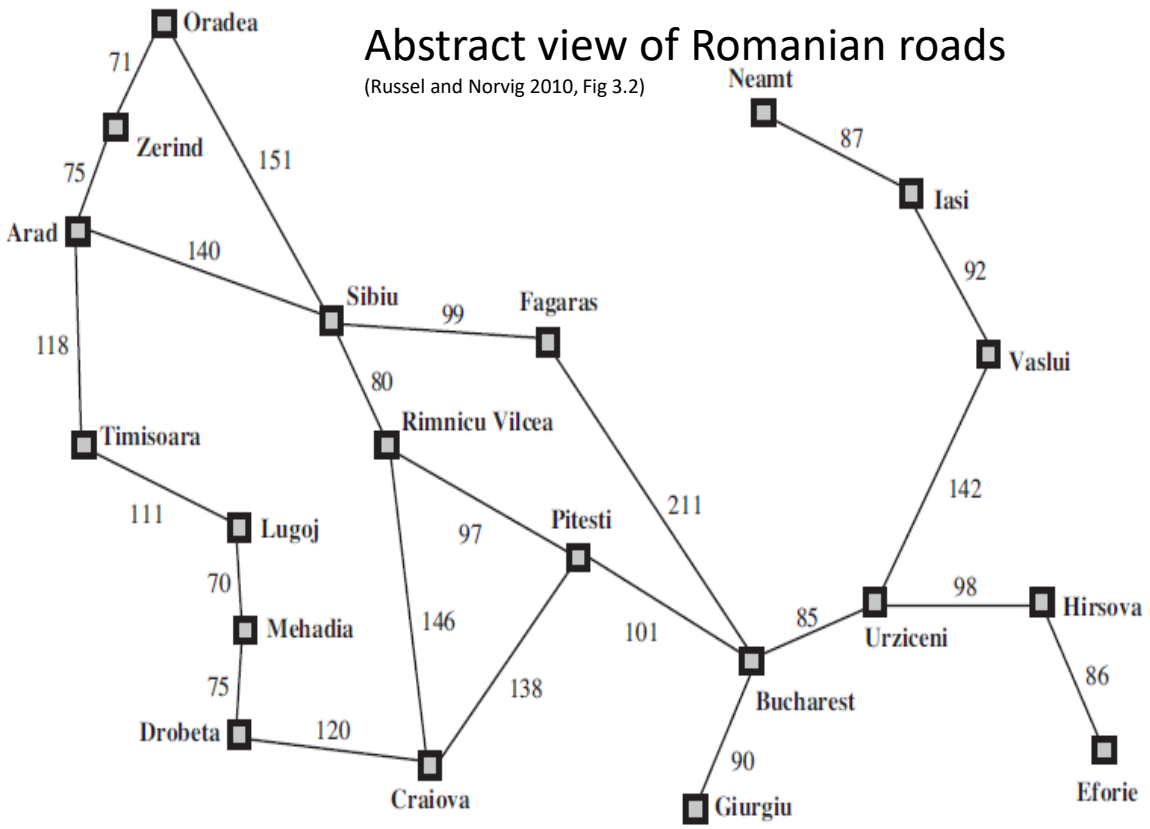
  $\forall n\ g'(n) = k \text{ and } h'(n) = -depth(n)$

- Non-optimal
- Incomplete search

- Why bother?

Abstract view of Romanian roads

(Russel and Norvig 2010, Fig 3.2)



Depth-first search (DFS)

- DFS will explore other paths when there are no successors.
- Fast! If you hit the right path… but the average case analysis is $O(b^m)$ where m is maximum depth.
- Space complexity is better: $O(bm)$

Iterative deepening

- Prevents infinite loops of depth-first search
- Basic idea
  - Depth-first search with a maximum depth
  - If the search fails, repeat with a deeper depth

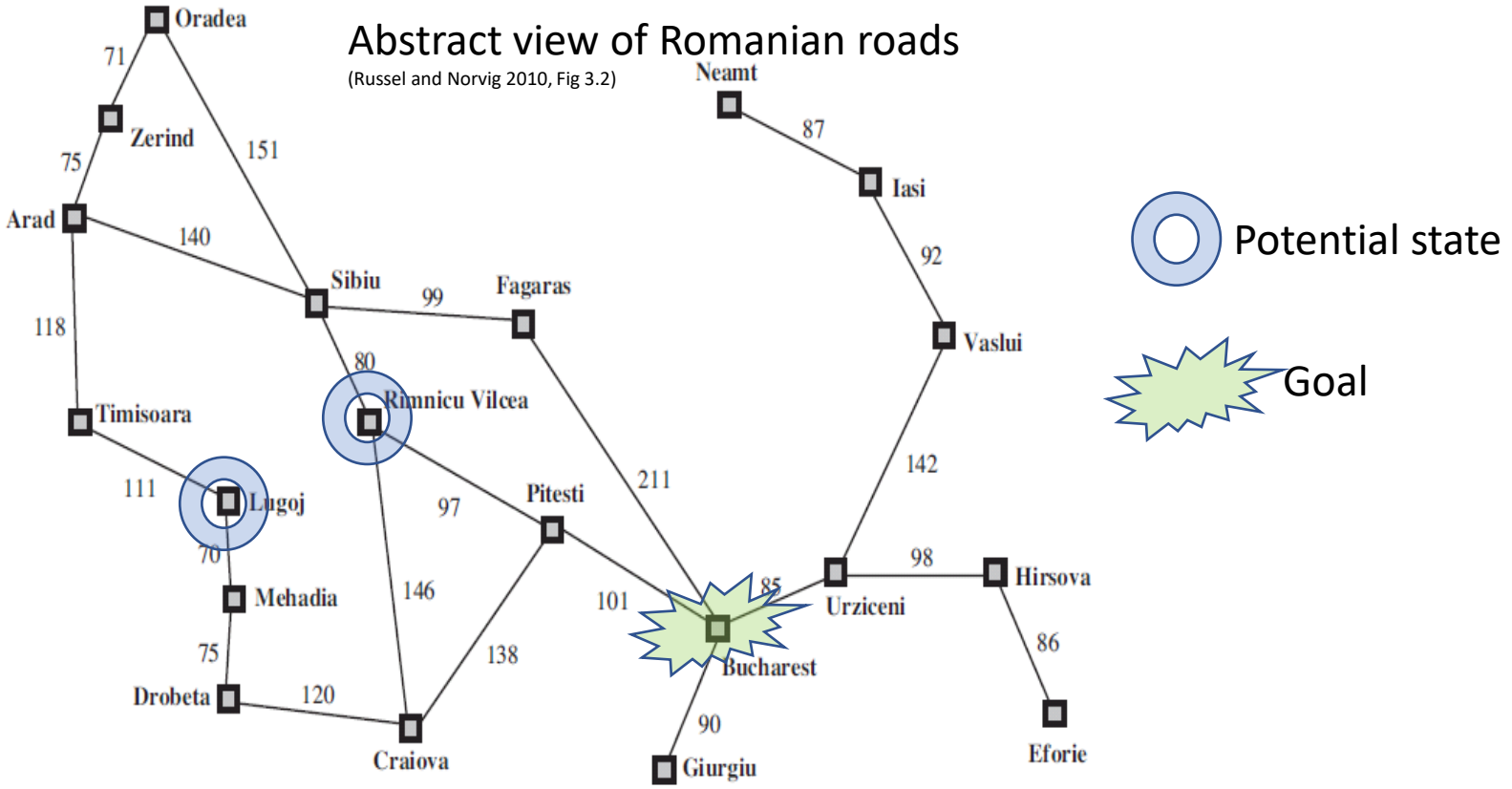
Uninformed search

- Other variants exist

- For large search spaces, uninformed search is usually a bad idea

Informed, or heuristic, search

- General idea: Can we guess the cost to a goal based on the current state?



Heuristic

- h(n) – Actual cost from a search graph node to a goal state *along the cheapest path*.

- h’(n) – An estimate of h(n), known as a heuristic.

Note that your text does not make a notational distinction between the actual cost and the estimated one and always uses h(n), so we will frequently follow suit.

Heuristic

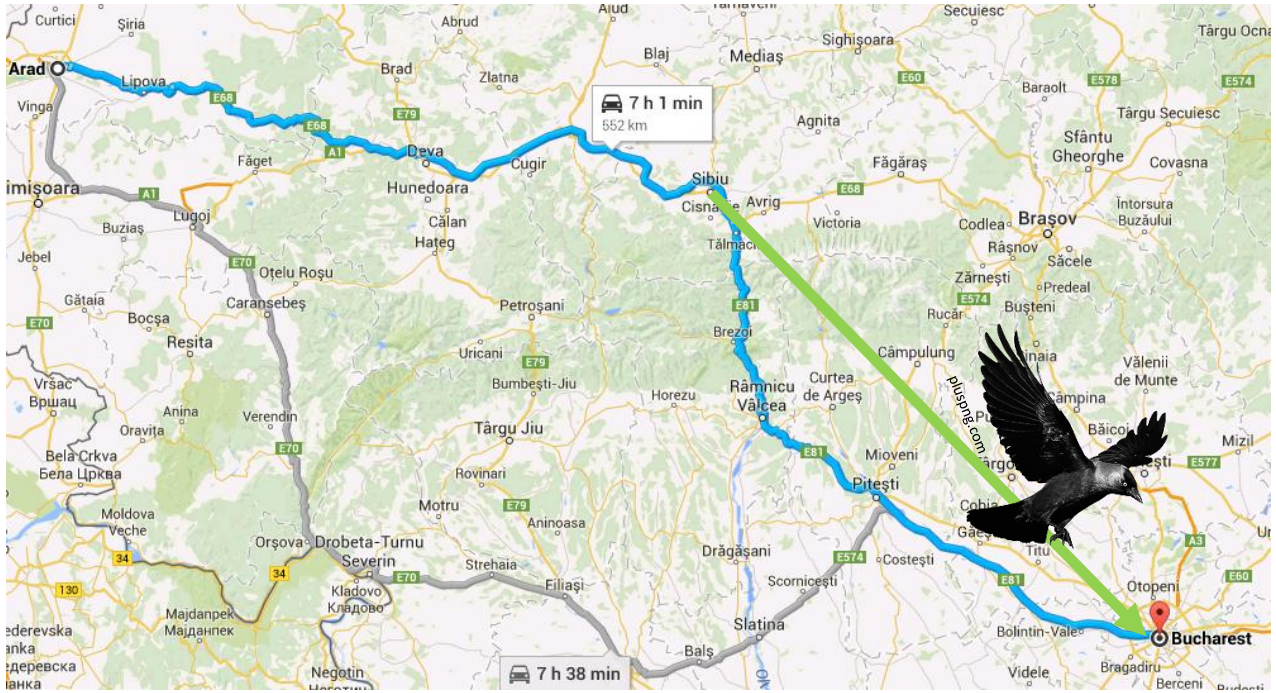
- $h(n)$ is always ≥ 0
- $h(n)$ is problem specific
- Estimators of $h(n)$ are similar.

- One can think of a heuristic as an educated guess.
  We will look at how to construct these later…

Greedy best-first search

- g(n) = 0, h(n) is heuristic value
- Example h(n) for Romania example:

  as the crow flies distance or straight line distance h_{SLD}



Greedy best-first search

- "Greedy"— at each step it tries to get as close to the goal as it can.
- expend node with shortest crow flies distance or straight line distance h_{SLD}

h'(n) = as the crow flies distance from problem state to goal state

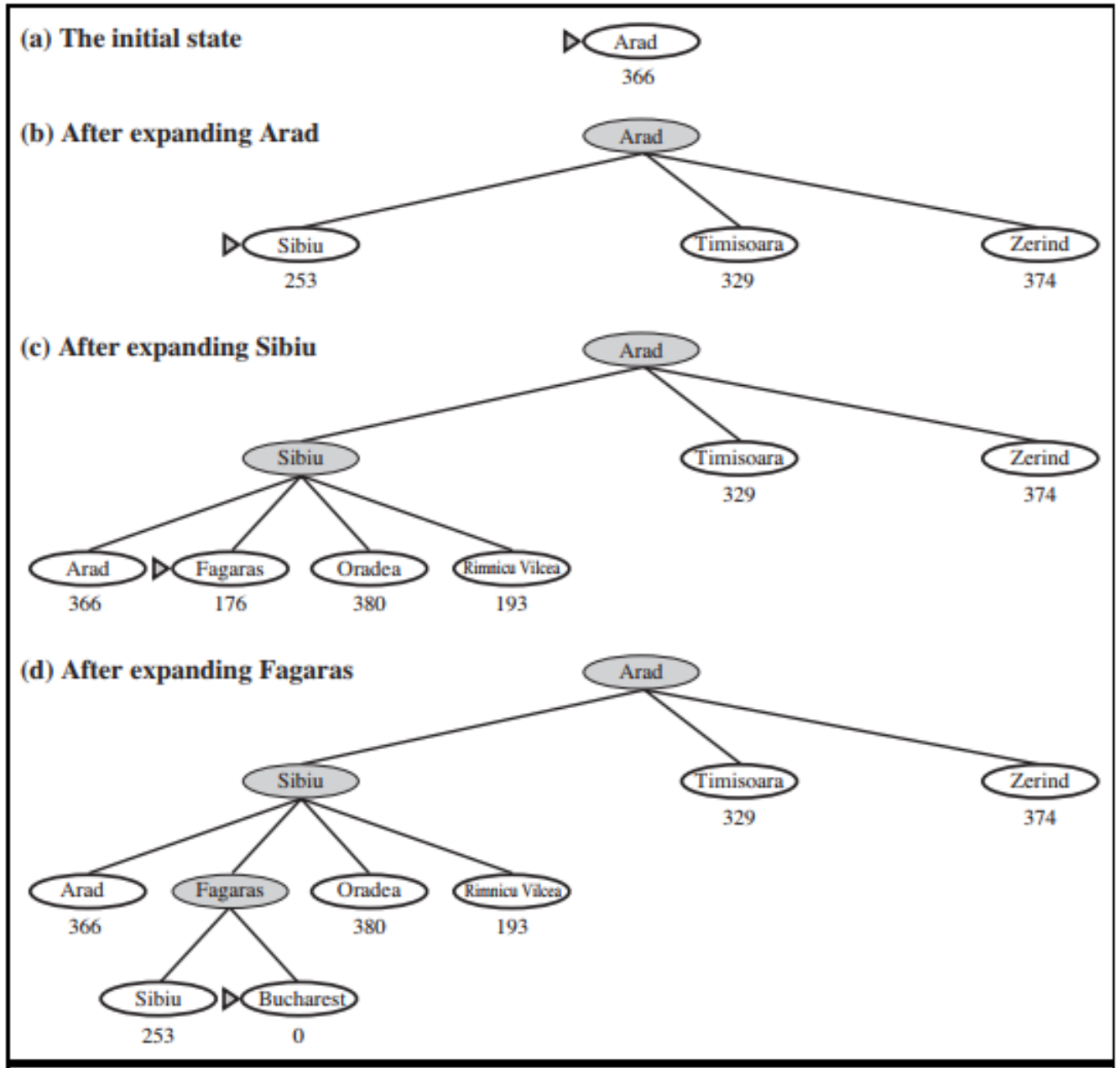


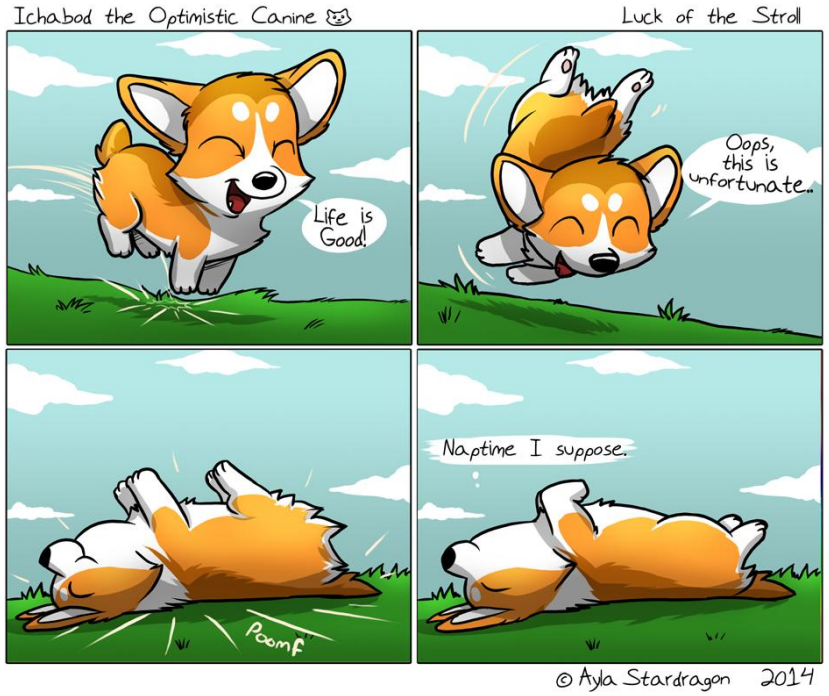
Figure 3.23 Stages in a greedy best-first tree search for Bucharest with the straight-line distance heuristic h_{SLD}. Nodes are labeled with their h-values.

A* Search

- “A-star” search uses:
  - g(n) = cost incurred to n
  - h(n) = estimate to goal

  A* is the estimated cost, f(n) = g(n)+h(n) from start to goal through state n

Heuristic properties



- admissible – h'(n) is optimistic:

$$h'(n) \leq h(n)$$

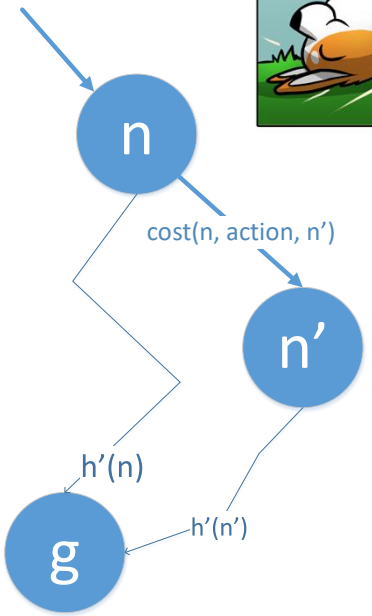
It *never overestimates* the cost to goal.

- consistency – h'(n) satisfies:

$$h'(n) \leq cost(n, action, n') + h'(n')$$

This is also known as monotonicity

crow flies distance is a consistent heuristic



Note: We are being careful about distinguishing the heuristic estimator h'(n) from the actual distance h(n)

Heuristic properties

- Every consistent heuristic is also admissible.
- Optimality of A* is guaranteed to be:
  - for trees
    A* optimal if h'(n) is admissible
    - $h'(n) \leq h(n)$
  - for graphs
    A* optimal if h'(n) is consistent
    - $h'(n) \leq cost(n, action, n') + h'(n')$

To find the shortest path from initial state to goal state and guid search towards the most promising path.

Understanding A* (A-star)

Remember: f(n) = g(n)+h'(n)

h'(n) = as the crow flies distance from problem state to goal state

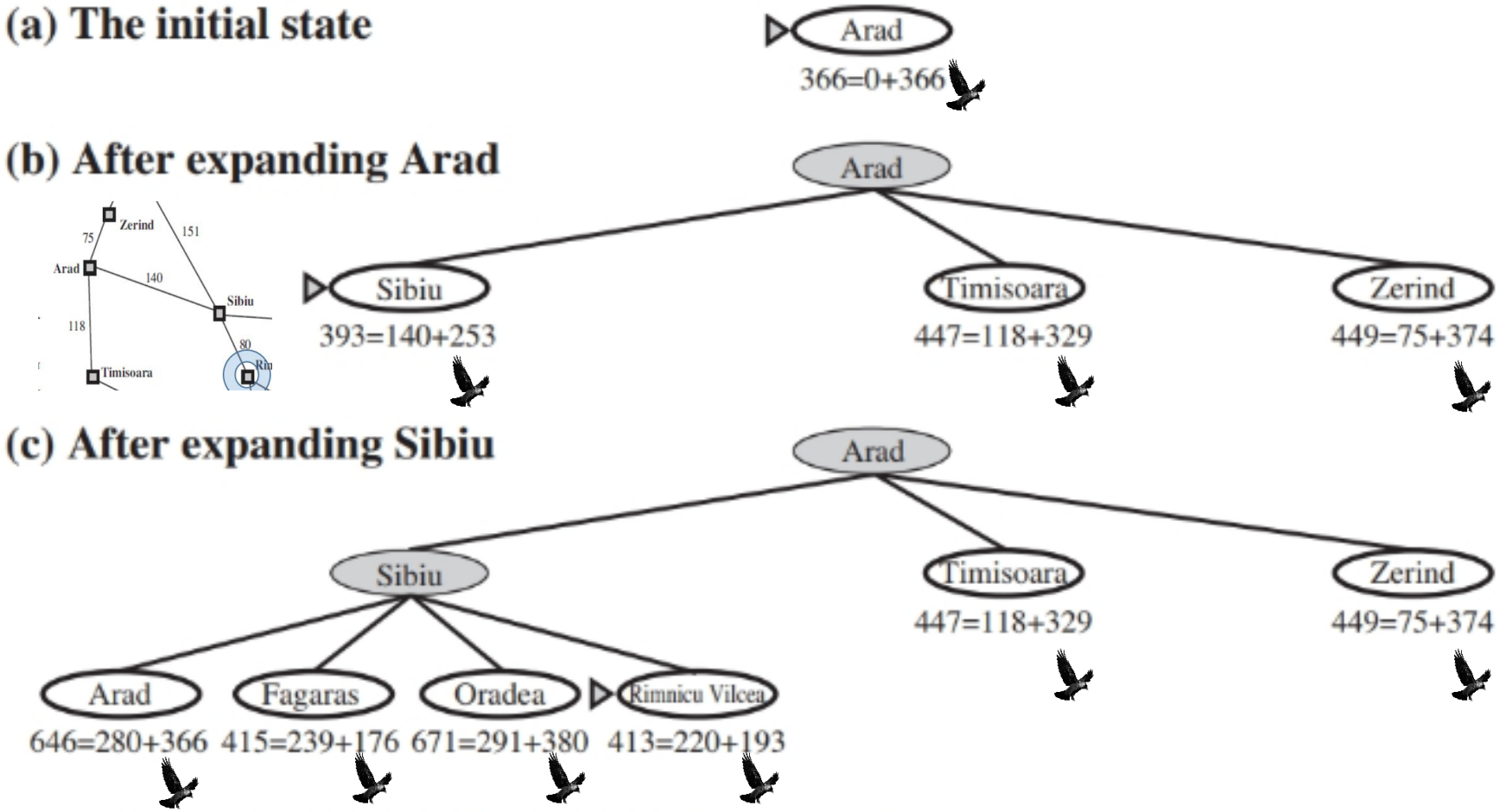
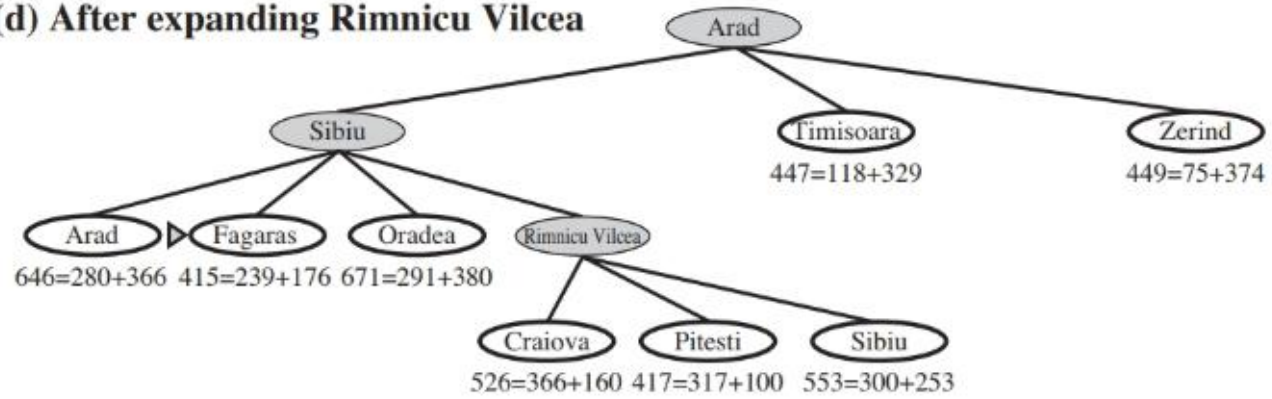
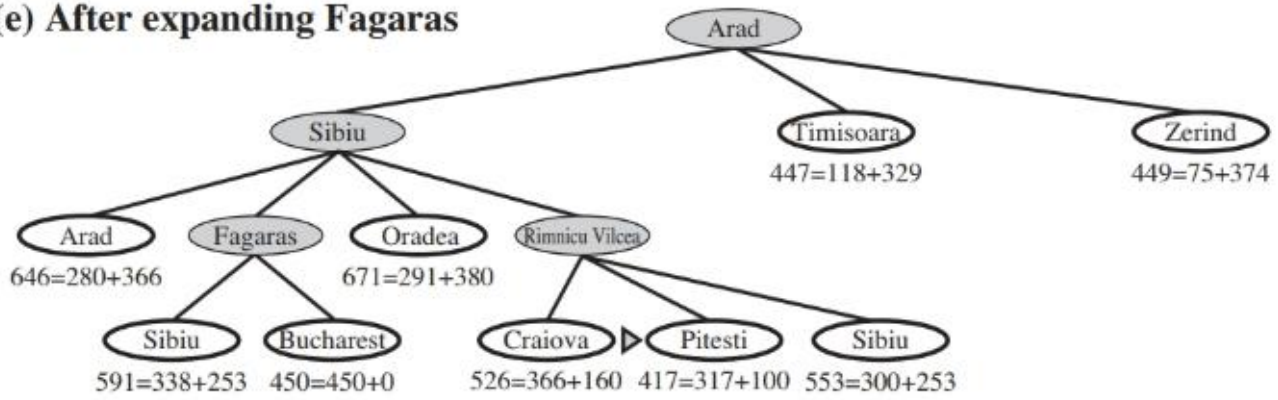


Figure 3.24, R&N

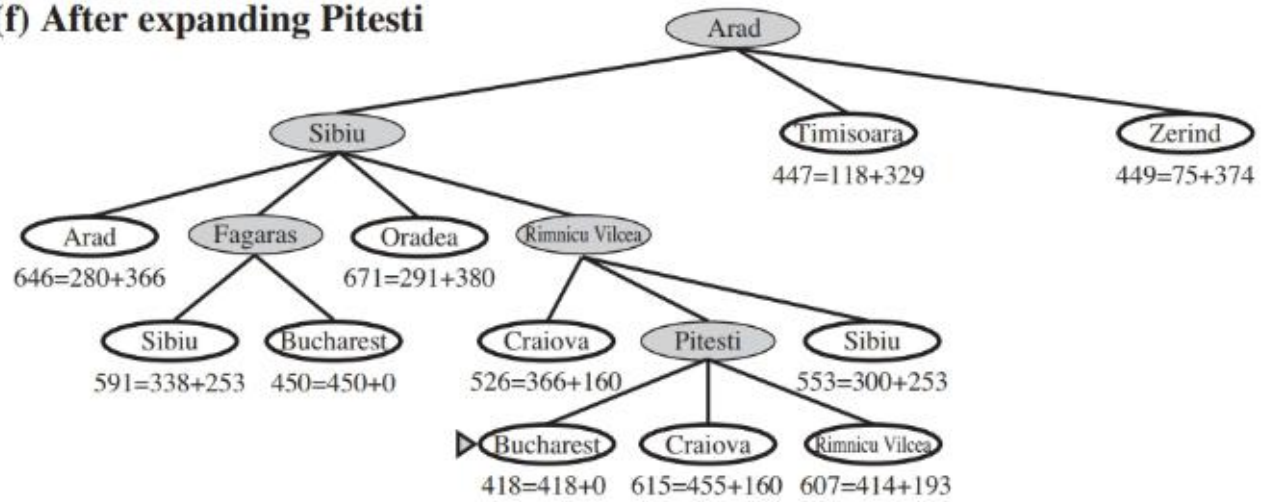
(d) After expanding Rimnicu Vilcea



(e) After expanding Fagaras



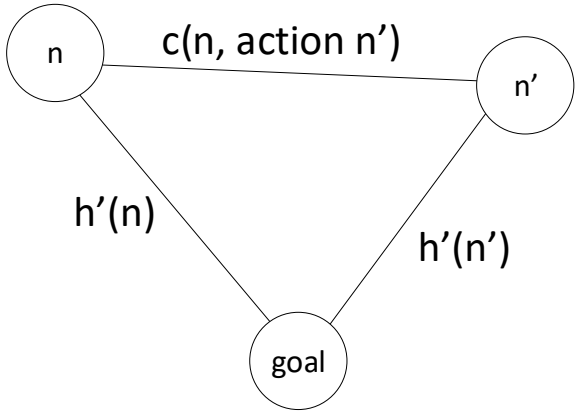
(f) After expanding Pitesti



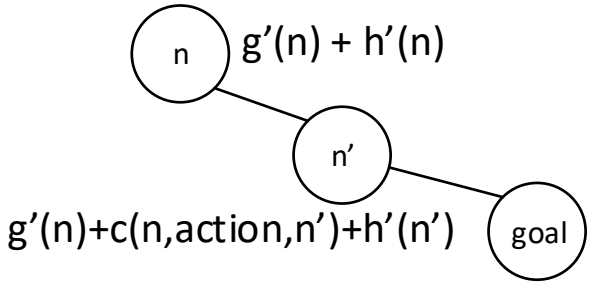
Understanding A* optimality

Consistency revisited:
the ▲ inequality – the sum of any two sides ≥ third side

$h'(n) \leq c(n,action,n')+h'(n')$



If h' consistent and costs are nonnegative, values of f(n) along any path are *nondecreasing*.



Understanding A* optimality

- When h(n) is consistent, the properties of:
  - nondecreasing values of f(n)
  - guarantee that we pick the best path to n

  ensure that the first goal node we find is optimal.

- Completeness holds when there are a finite number of nodes with f(n) < the optimal cost

h(n) – cost from n to least expensive goal

Limitations of A*

- Need to find a heuristic
- Want an optimal path? Show heuristic is
  - admissible (tree search) or
  - consistent (graph search).
- Want completeness?
  Show the graph is finite for nodes with cost lower than the cost of optimal solution path
- Note: expanded set requires nodes in memory (or at least cached) and is a frequent limitation of A*,
  - e.g., vacuum world with agent can clean up any square for unit cost without visiting it. N initial dirty state, 2^N states all are on optimal solution path

A* variants

- iterative deepening A*
  Same idea as iterative depth-first search,
  but we place limits on f(n)

- SMA* - simplified memory A*
  - When memory is full
    - drops worst frontier node (highest f(n))
    - stores that value in parent, and will only reconsider branch when everything looks worse than the stored value
  - Details beyond our scope

Heuristic search summary

- A* can still have problems with space complexity
  - iterative deepening A*
  - other alternatives listed in text

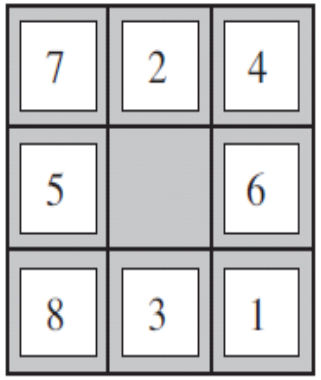
- Complexity of A* search is tricky, but is related to
  - the error in the heuristic, h(n)-h’(n)
  - and solution depth

Developing heuristics

- Requires
  - knowledge of problem domain
  - thinking a bit (usually)

- Effort to show that heuristic is
  - admissible
  - consistent

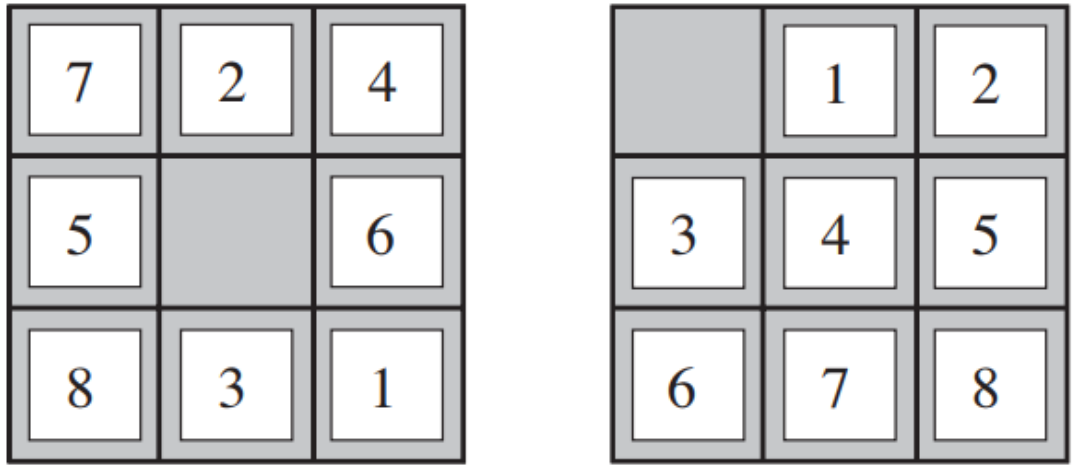
- What heuristics could we use for the N-puzzle?



Start State

N-puzzle heuristics

- Common heuristics
  - $h_1(n)$ – Number of misplaced tiles
  - admissible because any tile that is out of place must be moved at least once
  - $h_2(n)$ – Sum of Manhattan[1] distance of tiles to solution
  - admissible because all any move can do is move one tile one step closer to the goal



Start State Goal State

$$h_2 = 3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 = 18$$

[1] Also known as city-block distance, the sum of vertical and horizontal displacement.

Heuristics and performance

- Branching factor
  - Measured against a complete tree of solution depth d
  - Suppose A* finds a solution at
    - depth 5
    - 52 nodes expanded (53 with root)
  - A complete tree of depth 5 would have

$$52 + 1 = b^* + (b^*)^2 + (b^*)^3 + (b^*)^4 + (b^*)^5$$

    where b* is the branch factor
  - Using a root finder for

$$1\left(b^*\right)^5 + 1\left(b^*\right)^4 + 1\left(b^*\right)^3 + 1\left(b^*\right)^2 + 1\left(b^*\right)^1 - 53\left(b^*\right)^0 = 0$$

    we see b*≈1.92

Heuristics and performance

- 8-puzzle example averaged over 100 instances

IDS – Iterative deepening search

$h_1(n)$ – Number of misplaced tiles

$h_2(n)$ – Sum of Manhattan[1] distance of tiles to solution

depth of solution (d)

	Search Cost (nodes generated)			Effective Branching Factor		
d	IDS	$A^*(h_1)$	$A^*(h_2)$	IDS	$A^*(h_1)$	$A^*(h_2)$
2	10	6	6	2.45	1.79	1.79
4	112	13	12	2.87	1.48	1.45
6	680	20	18	2.73	1.34	1.30
8	6384	39	25	2.80	1.33	1.24
10	47127	93	39	2.79	1.38	1.22
12	3644035	227	73	2.78	1.42	1.24
14	–	539	113	–	1.44	1.23
16	–	1301	211	–	1.45	1.25
18	–	3056	363	–	1.46	1.26
20	–	7276	676	–	1.47	1.27
22	–	18094	1219	–	1.48	1.28
24	–	39135	1641	–	1.48	1.26

Fig. 3.29 R&N

- branch factors closer to one are better

Finding heuristics

- Okay, developing a heuristic is hard
- Can we make it easier?

Relaxed problem heuristics

- Let's return to the N-puzzle
- Suppose we allowed
  - A tile to move onto the next square regardless of whether or not it was empty.
  - A tile to move anywhere.
- These are relaxations of the rules

Relaxed problems

We can think of these as expanding the state space graph.

3		7
2	1	8
4	6	9

3	7	8
2	1	
4	6	9

3	7	
2	1	8
4	6	9

	3,7	
2	1	8
4	6	9

...many more...

3	7	
2		8
4	6	1,9

Relaxed problem heuristics

- The original state space is a subgraph of the new one.
- Heuristics on relaxed state space
  - Frequently easier to develop
  - If admissible/consistent properties hold in relaxed space, they also hold in the problem state space.

Relaxation

- Can specify problem in a formal language, e.g.
  - move(A,B) – means we can move A to position B
    We can do this if

    (verticalAdjacent(A,B) or horizontalAdjacent(A,B))

    and isempty(B)
- Possible relaxations
  - move(A,B) if adjacent(A,B)
  - move(A,B) if isempty(B)
  - move(A,B)

Automatically generated heuristics

With a formal specification of the problem there exist algorithms to find heuristics (beyond our scope, e.g. ABSOLVER)

Machine Learning, 12, 117–141 (1993)
© 1993 Kluwer Academic Publishers, Boston. Manufactured in The Netherlands.

Machine Discovery of Effective Admissible Heuristics

ARMAND E. PRIEDITIS PRIEDITIS@cs.UCDAVIS.EDU
Department of Computer Science, University of California, Davis, CA 95616

Multiple heuristics

- Regardless of how generated, one may develop multiple heuristics for a problem

- We can merge them

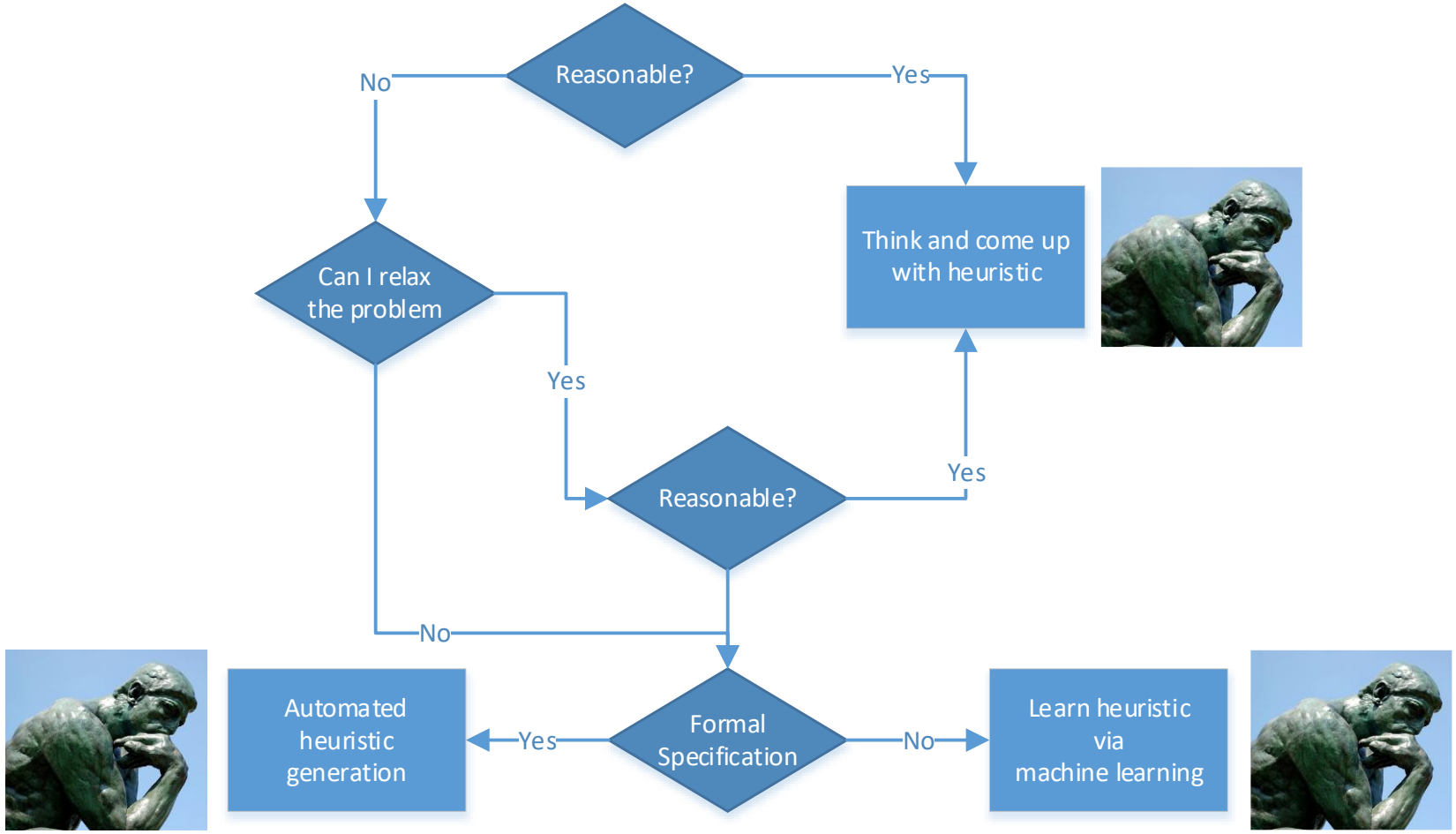
$$h'(n) = \max\left(h'_1(n), h'_2(n), \ldots, h'_i(n)\right)$$

why maximum?

Since component heuristics are admissible (never overestimate), maximum is most accurate

Learning heuristics

- Use experience to learn heuristics
- Beyond our reach for now… (machine learning)

Heuristic summary

(rough outline, no substitute for a little thought)